Evaluating Lightweight Machine Learning for Malicious Web Request Detection Under Group-Aware Splitting

Mohamed Youssef
School of Computing
Queen's University
Kingston, Canada
25tynh@queensu.ca

Mohanad Ghoraba
School of Computing
Queen's University
Kingston, Canada
25lhc5@queensu.ca

***Abstract*—This paper studies binary detection of benign and malicious web requests using classical machine learning. The experiment uses the openappsec WAF Comparison Dataset and keeps the evaluation grouped by source file instead of using a normal random split. After parsing, conservative cleaning, exact deduplication, and normalized deduplication, the final dataset contains 460,579 labeled requests. The models use character-level TF-IDF features together with a small set of request-structure features. Logistic Regression, Linear Support Vector Machine, Random Forest, and XGBoost are evaluated on a grouped holdout split and with grouped cross-validation. On the selected holdout split, XGBoost gives the best F1-score, 0.9735. Under grouped cross-validation, the ranking changes: Linear Support Vector Machine and Logistic Regression are more stable, both reaching mean F1 close to 0.964. The main finding is that the reported best model depends on the evaluation protocol, especially when related requests may share source-specific patterns.**

***Index Terms*—malicious web request detection, web application firewall, source-file grouping, character n-grams, evaluation leakage, cybersecurity machine learning**

I. Introduction

Web applications receive input through paths, parameters, cookies, headers, and request bodies. Attackers use the same input channels for SQL injection, cross-site scripting, command injection, path traversal, local file inclusion, Log4Shell-style probes, and similar request-level attacks. A web application firewall (WAF) can block many of these patterns, but the rule view is not always enough. Small changes in encoding, delimiters, keyword spelling, or payload layout can make two requests look different while preserving the same attack behavior.

This project does not try to replace a WAF. The goal is narrower: evaluate whether a small machine-learning detector can classify request strings while using an evaluation setup that is not too optimistic. In this setting, the split matters as much as the model. A random row split can be misleading because security datasets often contain repeated payload templates, crawled traffic from the same application, generated attack strings, or near duplicates. If one version of a request appears in training and a very similar one appears in testing, the model may look good for the wrong reason.

For that reason, the experiments separate data by `source_file`. Requests from the same file stay in the same partition. This is not perfect, but it is stricter than shuffling all rows together. It tests whether the classifier still works when the test examples come from source groups that were not seen during fitting. The same idea is also used in grouped cross-validation.

The paper makes three contributions. First, it gives a complete request-detection pipeline based on character n-grams and structural request features. Second, it reports the results of four common models under both a grouped holdout split and grouped cross-validation. Third, it shows that XGBoost is strongest on the selected holdout split, while Linear SVM and Logistic Regression are more stable across grouped folds. This distinction is important because a single high holdout score can hide weaknesses in generalization.

II. Related Work

Most practical web request defenses start from rules. The OWASP Top 10 describes common web application risks, and the OWASP Core Rule Set provides WAF rules for attacks such as SQL injection, cross-site scripting, path traversal, and protocol abuse [1], [2]. Rules are useful because an analyst can read them and update them quickly. The weakness is also clear: rule coverage depends on how well the rules match the current payload syntax. Encoding, mixed case, comments, split keywords, and nested parameters can reduce direct matches.

Classical machine learning has been used for malicious URL and request detection because URLs and request strings are short, structured text. Character n-grams are a natural representation because many attack indicators are local substrings rather than full words. For example, SQL fragments, encoded spaces, traversal markers, quotes, semicolons, and shell metacharacters may appear in different positions but still have similar local patterns. Linear models have been strong baselines for this type of high-dimensional sparse input, while Random Forest and gradient boosting can use compact numeric features and nonlinear interactions [3]–[7].

Deep models have also been proposed for short security strings. Saxe and Berlin introduced eXpose, a character-level convolutional model for malicious URLs, file paths, and registry keys [8]. URLNet learns URL representations using character- and word-level convolutional components [9]. These models are important because they reduce manual feature design and can learn sequential structure. However, they also require more tuning and are not automatically better for every dataset. In a student-scale experiment and in many WAF-adjacent settings, classical baselines are still worth testing before moving to larger neural models.

A separate line of work warns that evaluation bias is common in security machine learning. TESSERACT showed that malware classifiers can look much better when experiments ignore temporal and spatial separation [10]. Arp et al. also discuss practical mistakes in applying machine learning to computer security, including weak baselines, leakage, and unrealistic experimental assumptions [11]. These warnings are relevant here because web request datasets can contain repeated templates and source-specific artifacts. The present work follows the same basic caution: keep related examples out of opposite sides of the split whenever possible.

III. Problem Formulation

The task is binary supervised classification. Given a request string x_i, the model predicts $y_i \in \{0, 1\}$, where 0 represents benign traffic and 1 represents malicious traffic. The classifier outputs either a label or a score that can later be thresholded.

The errors are not equally convenient. A false negative means that a malicious request is missed. A false positive means that a benign request may be blocked, challenged, or sent for review. Accuracy is therefore not enough, especially because the test partition is highly imbalanced. The main metrics reported here are precision, recall, F1-score, PR-AUC, Matthews correlation coefficient (MCC) [14], and grouped cross-validation stability.

The grouping variable is part of the problem setup. Let g_i be the source file for request x_i. In the grouped holdout split, no g_i is allowed to appear in both training and testing. In grouped cross-validation, each fold follows the same rule. This makes the evaluation harder than a random row split and reduces the chance that the model is rewarded for memorizing file-specific request patterns.

IV. Dataset

The experiments use the openappsec WAF Comparison Dataset [12]. The original project material describes more than one million requests collected from production websites and malicious request files. The dataset includes benign browsing traffic and several attack categories. The raw data therefore contains a mixture of normal application requests, tracking URLs, encoded fields, long query strings, and attack payloads.

The notebook pipeline first parsed JSON request files and extracted the request strings. It then assigned the binary label and preserved `source_file` metadata for later grouping. Cleaning was conservative. The pipeline did not remove characters such as percent signs, quotes, slashes, parentheses, semicolons, repeated delimiters, or encoded substrings. In normal text classification these characters may look like noise. In this task they can be the evidence of an attack, so removing them would weaken the input.

TABLE I
Dataset Processing Stages

Stage	Rows	Benign	Malicious
Before cleaning	1,067,084	1,030,122	36,962
After cleaning	1,067,084	1,030,122	36,962
After deduplication	460,579	430,392	30,187

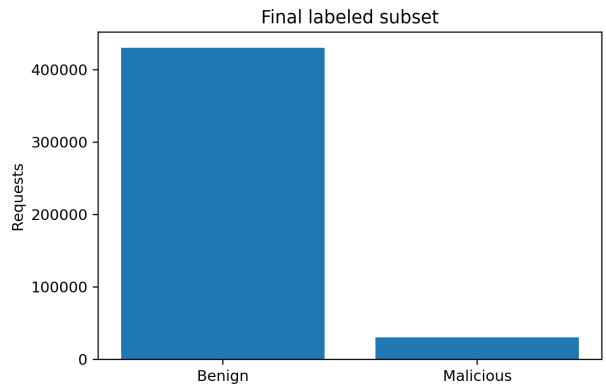

Fig. 1. Class distribution after cleaning and deduplication. The final subset is still mostly benign.

Table I shows the main reduction in dataset size. Exact deduplication removed identical requests. Normalized deduplication removed requests that became identical after normalization. This step is important because duplicate-heavy data can inflate results if copies of the same request pattern appear in both training and testing.

The final dataset contains 460,579 labeled requests, with 430,392 benign and 30,187 malicious examples. Fig. 1 shows this imbalance. The grouped holdout split is even more imbalanced, as shown in Table II. The test set has 84,059 requests but only 404 malicious requests. This means that a small change in false negatives can noticeably change recall and F1 on the holdout test set.

The holdout partition was produced with `GroupShuffleSplit`, test size 0.20, and random seed 42. Leakage checks found zero overlap in source files, exact request strings, and normalized request signatures between the training and test partitions.

V. Methodologies

The experiment follows four steps: preprocessing, grouped splitting, feature construction, and model evaluation. The order matters. Splitting and fitting were handled so that information from the test side did not enter the training process.

Preprocessing converted each raw record into a request string and a binary label. Rows with unusable request text

TABLE II
FINAL GROUPED TRAIN-TEST SPLIT

Partition	Rows	Benign	Malicious	Groups
Train	376,520	346,737	29,783	558
Test	84,059	83,655	404	140
Total	460,579	430,392	30,187	698

were removed only when necessary. Exact and normalized duplicates were then removed. The source-file field was kept throughout this stage because it was needed for splitting and cross-validation. The preprocessing was intentionally conservative: suspicious punctuation and encoded substrings were preserved instead of being cleaned away.

The split was grouped by source file because requests from the same file may share collection artifacts, repeated payload templates, application-specific query parameters, or crawler behavior. A random split would ignore this and could produce an easier test set. Grouping by file does not solve every possible leakage issue, but it directly addresses the most obvious source of related examples in this project.

All feature extractors and models were built inside pipelines. The TF-IDF vectorizer, scaler, SVD reducer for tree models, and classifier were fitted only on the training portion of the current split or fold. This avoids a common mistake where the vocabulary or dimensionality reducer is learned from the full dataset before cross-validation. That mistake can leak information about the test distribution even when labels are not used.

VI. MODELLING

The input representation combines character-level TF-IDF features with handcrafted request features. Character n-grams are useful because malicious request evidence is often local: `../`, `%20`, `union`, `select`, quotes, parentheses, shell separators, and encoded bytes. The TF-IDF analyzer used character n-grams of length 3 to 5 with minimum document frequency 3.

The handcrafted feature set measures request length and syntax shape. It includes request length, digit count, slash count, backslash count, question-mark count, ampersand count, equals-sign count, percent-sign count, dot-dot count, special-character count, whitespace count, dash count, underscore count, colon count, semicolon count, open-parenthesis count, close-parenthesis count, Shannon entropy, percent-encoded triplet count, suspicious keyword density, path depth, query-parameter count, digit ratio, special-character ratio, encoding ratio, and delimiter ratio.

Fig. 2 shows why the structural features should not be interpreted one at a time. In this dataset, benign requests have a higher mean length than malicious requests, probably because benign traffic contains tracking and application URLs with many parameters. The medians are closer. This supports using length together with encoding, delimiter, keyword, and character n-gram features.

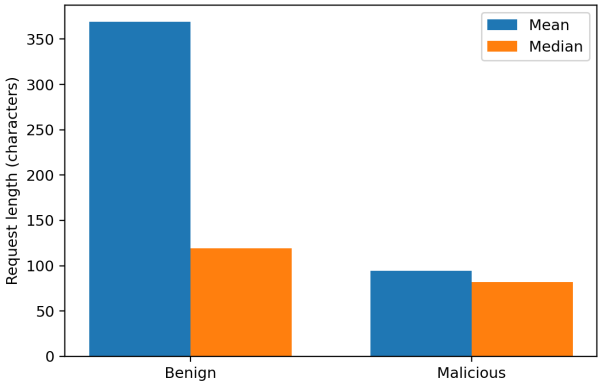

Fig. 2. Mean and median request length by class. Length is useful context, but it is not enough by itself.

Four models were tested: Logistic Regression, Linear SVM, Random Forest, and XGBoost. The two linear models used sparse TF-IDF features plus handcrafted features. The tree models used a smaller TF-IDF representation followed by truncated SVD, because the tree implementations used here are more practical with a dense reduced representation.

The exact settings are listed in Table III. These values were kept fixed for the reported comparison. The purpose was not to exhaustively tune every model, but to compare reasonable compact classifiers under the same grouped evaluation discipline.

VII. RESULTS

Table IV reports the main results. On the grouped holdout split, XGBoost performs best with F1 = 0.9735, PR-AUC = 0.9965, and MCC = 0.9736. Random Forest is close, with perfect holdout precision but lower recall. Linear SVM has weaker precision than the tree models, but it misses fewer malicious requests. Logistic Regression has the lowest holdout precision, although its recall is also high.

The holdout numbers should be read with the class distribution in mind. The test set contains 404 malicious requests. For XGBoost, 19 false negatives correspond to recall of 0.9530. For Linear SVM and Logistic Regression, the false negatives are only 6 and 5, but this comes with more false positives and lower precision. In other words, the holdout split is not large enough in the malicious class to treat small differences as final proof of one model being generally better.

Fig. 3 gives the XGBoost confusion matrix at the default threshold. The model correctly classifies 83,653 benign requests and 385 malicious requests, while producing 2 false positives and 19 false negatives. This is a strong holdout result, but it is still only one grouped split.

Grouped cross-validation changes the conclusion. Linear SVM and Logistic Regression both reach mean F1 close to 0.964 and have much smaller F1 standard deviations than the tree models. XGBoost and Random Forest keep very high precision, but their mean recall drops to 0.6780 and 0.6574.

TABLE III
PIPELINE CONFIGURATION USED IN THE EXPERIMENTS

Component	Configuration
Random seed	42
Holdout split	`GroupShuffleSplit`, test size = 0.20
Grouped cross-validation	`GroupKFold`, 3 folds
TF-IDF analyzer	Character-level
TF-IDF n-grams	3 to 5
TF-IDF minimum document frequency	3
Linear-model TF-IDF maximum features	2,500
Tree-model TF-IDF maximum features	1,500
Tree dimensionality reduction	`TruncatedSVD`, 80 components
Logistic Regression	`liblinear`, $C = 1.0$, maximum iterations = 600, class weight = balanced
Linear SVM	$C = 0.75$, class weight = balanced
Random Forest	100 trees, maximum depth = 18, minimum samples per leaf = 3, maximum samples = 0.7, balanced subsample class weights
XGBoost	100 estimators, maximum depth = 4, learning rate = 0.10, subsample = 0.80, column sample by tree = 0.80, regularization lambda = 1.0, tree method = hist

TABLE IV
PERFORMANCE SUMMARY ACROSS EVALUATION PROTOCOLS

	Holdout Test Set						Grouped Cross-Validation			
Model	Prec.	Rec.	F1	PR-AUC	MCC	FN	Mean Prec.	Mean Rec.	Mean F1	F1 Std.
XGBoost	0.9948	0.9530	0.9735	0.9965	0.9736	19	0.9984	0.6780	0.7890	0.1764
Random Forest	1.0000	0.9406	0.9694	0.9944	0.9697	24	0.9970	0.6574	0.7724	0.1815
Linear SVM	0.8864	0.9851	0.9332	0.9521	0.9342	6	0.9988	0.9350	0.9644	0.0487
Logistic Regression	0.7793	0.9876	0.8712	0.7573	0.8767	5	0.9977	0.9359	0.9640	0.0538

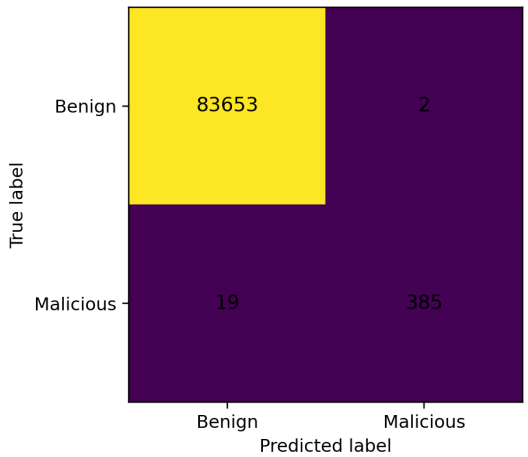

Fig. 3. XGBoost confusion matrix on the grouped holdout split at the default threshold.

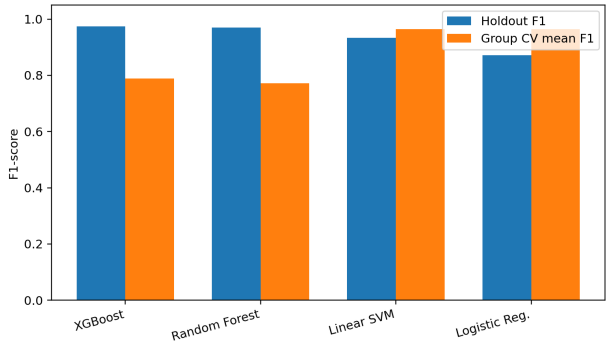

Fig. 4. The preferred model changes when moving from the selected grouped holdout split to grouped cross-validation.

This suggests that the tree models fit the selected holdout split well but are less stable across different source-file partitions.

Fig. 4 summarizes this ranking change. The point is not that XGBoost is unusable. The point is that its holdout advantage does not transfer consistently across grouped folds. For a detector that will see new traffic sources, this difference matters.

The notebook also tested score thresholds for XGBoost. A threshold of 0.15 gave the best holdout F1 among the tested thresholds, increasing F1 to 0.9864. This result is useful for understanding the precision-recall trade-off, but it is not an unbiased final estimate because the threshold was selected using holdout outputs.

VIII. COMPARISON

The comparison depends on the decision criterion. If the criterion is the selected holdout split, XGBoost is the strongest model. It has the highest holdout F1, PR-AUC, and MCC, and only two false positives. That makes it attractive when false positives are expensive and the traffic distribution is close to the selected test split.

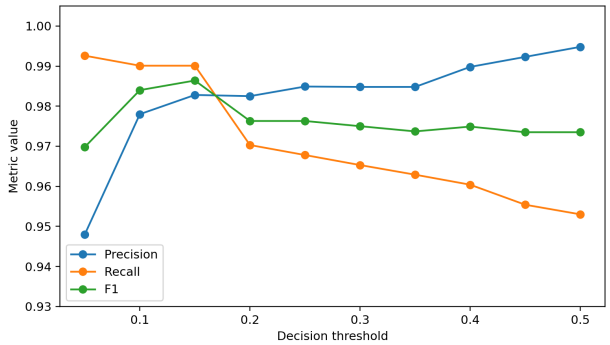

Fig. 5. XGBoost holdout precision, recall, and F1 across tested decision thresholds.

TABLE V
PRACTICAL INTERPRETATION OF MODEL BEHAVIOR

Model	Holdout	Grouped CV	Complexity
XGBoost	Best	Weak	Medium
Random Forest	Strong	Weak	Medium
Linear SVM	Moderate	Strong	Low
Logistic Regression	Weak	Strong	Low

If the criterion is consistency across source-file groups, Linear SVM and Logistic Regression are better choices. Their holdout precision is weaker, but their grouped cross-validation recall and F1 are much more stable. This is a reasonable trade-off in security work because missing attacks from a new source group can be worse than accepting more analyst review on suspicious benign requests.

Table V gives the practical summary. Fig. 1, Fig. 2, Fig. 3, Fig. 4, and Fig. 5 are referenced here because they explain different parts of the result: imbalance, feature behavior, error counts, ranking change, and threshold sensitivity. The tables provide the reproducible configuration and the exact metric values.

Table VI shows the patterns seen in representative XGBoost errors. The false positives were mostly benign requests with dense query syntax, such as tracking or search URLs. The false negatives were more often encoded SQL-style payloads. This is a useful failure mode because it points to concrete future work: better handling of encoded SQL variants and not just larger models.

IX. DISCUSSION

The result is not a simple model leaderboard. XGBoost is the best model on the selected holdout split, but Linear SVM and Logistic Regression are more reliable when the grouping is repeated across folds. This is exactly why grouped evaluation was used. Without it, the paper would probably overstate the tree-based result.

The large difference between holdout and cross-validation behavior may come from source-file structure. Tree models can exploit interactions among reduced features and handcrafted features. That can help when the test groups resemble parts

TABLE VI
REPRESENTATIVE ERROR PATTERNS FROM XGBOOST HOLDOUT ERRORS

Pattern	False Positives	False Negatives
`%`	1	19
`=`	2	19
`?`	2	19
`select`	0	17
`0x`	0	9
`union`	0	3
`cmd`	0	1

of the training distribution, but it can also make the model sensitive to group-specific patterns. Linear models are less flexible, which may be why they lose precision on the selected holdout split but keep stronger recall across folds.

The results also show that feature design still matters. Character n-grams capture local attack syntax, while hand-crafted features describe the shape of the request. Neither family is enough on its own. Benign URLs can be long and punctuation-heavy, and malicious payloads can be short or strongly encoded. The model needs both lexical fragments and structural indicators to avoid treating every complicated URL as malicious.

Another point is that the grouped split is stricter but still not perfect. Source files are only a proxy for independence. Two different files may still contain related payload generators, and one file may contain a mixture of easy and hard examples. Even with that limitation, grouping is a better choice than random splitting because it directly blocks the simplest form of duplicate or template leakage. For this project, the zero overlap checks on source files, exact requests, and normalized signatures are important parts of the reported result, not just preprocessing details.

The error patterns also give a useful sanity check on the model behavior. The false positives are not random strings; they look like real benign URLs with many delimiters and parameters. The false negatives are not simple obvious attacks; many include encoded SQL-like terms. This means the model is making plausible mistakes. That is better than seeing errors that contradict the task definition, but it also shows that detection cannot be judged only by aggregate metrics. A small table of errors is less impressive than a large score, but it is often more useful for understanding what the detector learned.

X. DEPLOYMENT CONSIDERATIONS

The classifier should be used as an additional scoring layer, not as a direct replacement for a WAF. Existing rules are still useful for known attacks and for clear policy enforcement. The model can add a probability score or secondary label for requests that are not clearly handled by rules.

Threshold selection should use a separate validation set, not the final test set. The threshold experiment in Fig. 5 is exploratory because it uses holdout outputs. A real deployment should also monitor drift, false positives, missed attacks, and changes in request structure. Retraining should keep grouped

validation so that repeated templates or source artifacts do not leak across partitions again.

Based on these experiments, Linear SVM is the safer starting point when stable grouped performance is the main concern. XGBoost is reasonable when the team cares most about the selected holdout behavior and can monitor drift closely. In both cases, analyst feedback should store the original request, model version, threshold, and final action so that later retraining remains auditable.

XI. Limitations

This study has several limitations. First, it uses one main dataset. The results are useful for this benchmark, but they should not be treated as universal web-traffic results. Second, the grouped holdout split produced a very small malicious test subset: 404 malicious requests out of 84,059 total test requests. This makes the holdout recall and F1 sensitive to a small number of false negatives.

Third, threshold tuning on the holdout split is exploratory. It helps show how the operating point changes, but it is not an unbiased final estimate. A separate threshold-validation partition is needed. Fourth, the paper does not compare against production WAF rules, deep neural baselines, or a multi-dataset benchmark. Those comparisons would be needed before making stronger claims about deployment performance.

There are also limits in the labeling setup. The model learns from request strings and source labels already present in the dataset. It does not verify exploit success, application context, response behavior, or whether a payload was blocked by an existing defense. Therefore, the results should be read as request-string classification results, not as a full measurement of application compromise risk.

A final limitation is that the experiment uses fixed feature settings and fixed model settings. That is acceptable for a controlled comparison, but it is not the same as a full hyperparameter study. More tuning might improve one of the weaker models, especially under grouped validation. The reported values should therefore be read as the outcome of this pipeline and configuration, not as the absolute ceiling for each algorithm.

XII. Conclusion

This paper evaluated malicious web request detection using grouped train-test separation and grouped cross-validation. The pipeline kept attack-relevant punctuation and encoding, removed exact and normalized duplicates, used source-file groups to reduce leakage, and compared four compact classifiers.

The main result is direct. XGBoost was strongest on the selected grouped holdout split, with F1 = 0.9735, PR-AUC = 0.9965, and MCC = 0.9736. However, the ranking changed under grouped cross-validation. Linear SVM and Logistic Regression were more stable, both reaching mean F1 close to 0.964. This means the evaluation protocol changes the practical recommendation.

Future work should test the same pipeline on more datasets, use a separate validation split for threshold selection, compare against production WAF rules and neural baselines, and study encoded SQL-style false negatives in more detail. A next version should also report per-attack-family metrics and compare file-based grouping with time-based testing when timestamps are available. Time-based testing may be closer to deployment because a detector is usually trained on older traffic and then used on newer traffic. The main takeaway is simple: for cybersecurity datasets, controlling leakage is not a formatting detail; it can change the result.

References

[1] OWASP Foundation, "OWASP Top 10: 2021," 2021. [Online]. Available: https://owasp.org/Top10/2021/

[2] OWASP Foundation, "OWASP ModSecurity Core Rule Set," 2024. [Online]. Available: https://owasp.org/www-project-modsecurity-core-rule-set/

[3] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: A library for large linear classification," *Journal of Machine Learning Research*, vol. 9, pp. 1871–1874, 2008. [Online]. Available: https://www.jmlr.org/papers/volume9/fan08a/fan08a.pdf

[4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. [Online]. Available: https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf

[5] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. [Online]. Available: https://projecteuclid.org/journals/annals-of-statistics/volume-29/issue-5/Greedy-function-approximation-A-gradient-boosting-machine/10.1214/aos/1013203451.full

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. [Online]. Available: https://arxiv.org/pdf/1603.02754

[7] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf

[8] J. Saxe and K. Berlin, "eXpose: A character-level convolutional neural network with embeddings for detecting malicious URLs, file paths and registry keys," arXiv:1702.08568, 2017. [Online]. Available: https://arxiv.org/abs/1702.08568

[9] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," arXiv:1802.03162, 2018. [Online]. Available: https://arxiv.org/abs/1802.03162

[10] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Security Symposium*, 2019, pp. 729–746. [Online]. Available: https://www.usenix.org/system/files/sec19-pendlebury.pdf

[11] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988. [Online]. Available: https://www.usenix.org/system/files/sec22summer_arp.pdf

[12] openappsec, "WAF Comparison Project," 2024. [Online]. Available: https://docs.openappsec.io/references/waf-comparison-project

[13] scikit-learn developers, "Cross-validation iterators with groups," scikit-learn documentation, 2024. [Online]. Available: https://scikit-learn.org/stable/modules/cross_validation.html

[14] scikit-learn developers, "matthews_corrcoef," scikit-learn documentation, 2024. [Online]. Available: public documentation